

온디바이스 sLLM을 위한 K26-Memory FPGA 후보 구조의 설계 공간 평가

Design-Space Evaluation of a K26-Memory FPGA Candidate Architecture for On-Device sLLMs

최윤혁 (CHOI YUNHYUK) · 한국디지털미디어고등학교 · ORCID 0009-0006-3537-0249

2026-08-01

초록

온디바이스 소형 언어 모델은 서버급 가속기를 사용하기 어려워 메모리 용량과 가중치 공급 대역폭을 제한된 전력·비용 안에서 함께 해결해야 한다. 본 연구는 Kria K26이 연산·제어를 맡고 외부 Memory FPGA가 다채널 가중치 공급을 맡는 확장형 후보 구조를 설계했다. 정적 작업 배정으로 일부 연산 클러스터가 유휴 상태에 빠질 때는 데이터 이동 비용을 고려한 작업 훔치기(Work Stealing)를 선택적으로 사용한다. 실제 Gemma 3 1B 그래프의 7,837개 노드에서 토큰당 183개 투영 연산을 추출하고, 이를 의존성이 있는 802개 출력 타일로 변환해 네 가지 초기 배치와 세 가지 대기열 정책을 비교했다. 합성 불균형 부하에서 지역성 인식 정책은 정적 배치보다 TileJob p95를 19.13% 줄이고, 단순 작업 훔치기보다 비지역 가중치 이동량을 35.49% 줄였다. 그러나 Gemma 형상에서는 초기 배치에 따라 p95 변화가 +0.28%에서 -19.79%까지 달라졌으며, K26 로컬 메모리 기준선도 외부 6.4 GB/s 후보보다 짧았다. 따라서 외부 8GB와 작업 재분배는 Gemma 1B의 필수 구성요소가 아니라 더 큰 모델·긴 문맥·로컬 경합이 확인될 때 채택할 확장 수단이다. 결과는 제한된 RTL 시뮬레이션과 분석 모델에 근거하며 보드 실측값이 아니다.

핵심어: 온디바이스 sLLM, Kria K26, Memory FPGA, 다채널 메모리, 작업 훔치기, SpinalHDL

Abstract

On-device small language models must address memory capacity and weight-delivery bandwidth within tighter power and cost limits than server accelerators. This work evaluates an extensible candidate architecture in which a Kria K26 handles control and computation while an external Memory FPGA supplies weights through multiple memory channels. Locality-aware work stealing is enabled only when static placement leaves compute clusters idle and its expected benefit exceeds data-movement cost. From the 7,837-node Gemma 3 1B graph, 183 projections

per token are mapped to 802 dependency-aware output tiles and evaluated under four initial placements and three queue policies. On a synthetic skew workload, the locality-aware policy reduces TileJob p95 by 19.13% against static placement and non-local weight movement by 35.49% against oldest-job stealing. On Gemma-shaped tiles, however, the p95 effect ranges from +0.28% to -19.79% depending on initial placement, and the modeled K26-local baseline remains shorter than the external 6.4-GB/s candidate. External 8GB memory and work stealing are therefore conditional expansion mechanisms rather than requirements for Gemma 1B. The results are based on bounded RTL simulation and analytical models, not board measurements.

Keywords: on-device sLLM, Kria K26, Memory FPGA, multi-channel memory, work stealing, SpinalHDL

I. 서론

온디바이스 환경에서는 서버급 GPU의 전력과 가격을 그대로 사용할 수 없다. 모델이 메모리에 들어가는지만 보는 것도 충분하지 않다. 연산부가 가중치를 기다리는 시간, 운영체제·활성값·KV cache가 쓰는 로컬 메모리와 가중치 공급의 경합, 여러 연산 클러스터 사이의 부하 균형이 함께 성능을 결정한다.

본 연구는 처음에 외부 메모리가 Gemma 3 1B의 용량 때문에 필요하다고 가정했다. 그러나 INT8 가중치와 32K 문맥의 용량을 계산한 결과 2.43 GiB로, 명목상 K26 로컬 4GB에 들어갔다. 이에 연구 질문을 “외부 메모리가 필요한가”에서 “가중치 공급을 로컬 실행 메모리와 분리하는 구조는 어떤 조건에서 이득인가”로 바꾸었다.

제안 후보는 K26의 연산·제어와 Memory FPGA의 다채널 가중치 공급을 분리한다. K26에는 네 연산 클러스터와 독립 대기열을 두고, 외부 쪽에는 네 DDR3L x16 채널과 네 논리 링크를 둔다. 정적 배정이 데이터 지역성을 지키더라도 작업이 한쪽에 몰리면 일부

클러스터가 높을 수 있다. 이때만 유틸리티 클러스터가 다른 대기열의 작업을 가져오며, 이미 배치된 가중치와 활성값을 옮기는 비용을 함께 계산한다.

본 연구의 기여는 다음과 같다. 첫째, 로컬 실행 메모리와 외부 가중치 공급 메모리의 역할을 분리한 온디바이스 확장 후보를 제시하고 K26-local 기준선과 비교했다.

둘째, 원격 복사 시간을 실제 링크 서비스에 부과해 초기 배치와 작업 재분배의 효과를 분리했다. 셋째, 실제 Gemma 그래프의 형상에서 TileJob을 만들고, 제한된 실제 가중치 타일이 DMA 명령·메모리 응답 경계·논리 링크·MatVec까지 통과하는 재현 가능한 흐름을 구현했다.

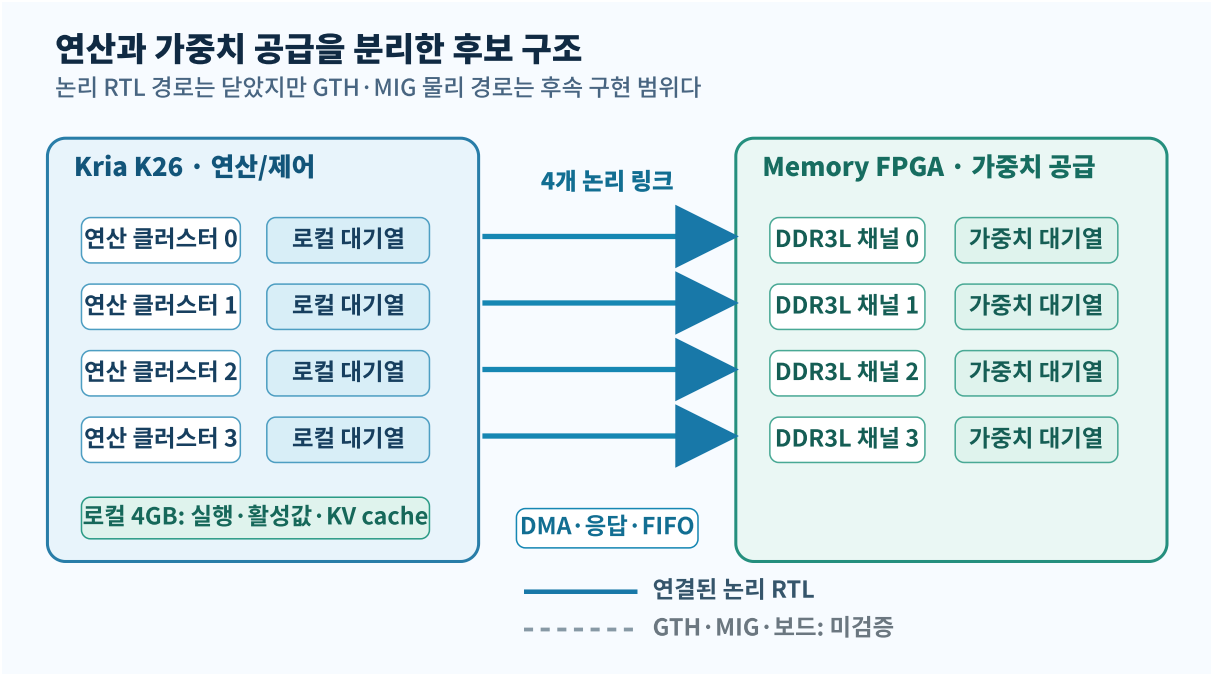


그림 1. K26의 연산·제어와 Memory FPGA의 가중치 공급을 분리한 후보 구조. 굵은 실선은 연결된 논리 RTL 경로이며, 점선의 GTH·MIG는 후속 물리 구현 범위다.

II. 관련 연구와 연구 공백

PagedAttention[1]은 KV cache의 페이지 관리 문제를, FlashAttention[2]은 attention 연산의 데이터 이동을 줄이는 방법을 다룬다. GPTQ[3], SmoothQuant[4], AWQ[5]는 모델 크기와 양자화 정확도의 균형을 연구한다. 이 연구들은 메모리 이동이 중요하다는 근거를 제공하지만, 여러 로컬 대기열과 외부 가중치 공급 링크를 함께 가진 소형 FPGA 구조의 초기 배치 문제를 직접 다루지 않는다.

FTRANS[6], DFX[7], FlightLLM[8]은 Transformer의 FPGA 매핑과 다중 FPGA 실행 구조를 제시했다. 이들은 완성된 매핑 흐름이나 처리량을 중심으로 평가한다. 본 연구는 최고 성능을 경쟁하지 않는다. 대신 K26 로컬 메모리와 외부 다채널 메모리 후보를 같은 작업 형상에서 비교하고, 작업 재분배가 가중치 복사 비용을 감수할 조건을 찾는다.

고전적 작업 흠치기[15]는 의존 작업의 부하 균형을 설명하고, LAWS[16]는 NUMA 환경에서 원격 메모리 비용을 고려한다. 본 연구의 정책은 이들보다 이론적으로 새롭다고 주장하지 않는다. 연구 공백은 실제 sLLM 투영

형상, FPGA 메모리 분리 구조, 초기 배치, 비지역 전송 비용을 하나의 공개 설계 흐름에서 비교한 자료가 부족하다는 점이다.

III. 온디바이스 가속기의 설계 목표

대상은 3B 이하 sLLM을 향한 확장형 후보이며, 이번 평가는 Gemma 3 1B를 사용한다. K26 로컬 4GB는 운영체제와 실행 환경, 활성값, KV cache, 자주 쓰는 데이터를 담당한다. 외부 메모리는 상주 가중치, 다음 층 사전 적재, 용량 확장을 담당하도록 역할을 나눈다.

표 1. 설계 목표와 이번 평가의 범위

항목	목표	이번 평가
연산	K26의 다중 연산 클러스터	4개 signed-INT8 MatVec 클러스터
로컬 메모리	실행 환경·활성값·KV cache	4.8~14.4 GB/s 유틸리티 대역폭 민감도
외부 메모리	8GB 이상, 채널·rank 확장	4×DDR3L x16, 3.2~12.8 GB/s 분석
링크	Memory FPGA→K26 가중치 전송	4개 논리 링크, 64~256 bit 민감도
작업 배치	지역성과 부하 균형	정적·라운드로빈·크기 균형·채널 친화
물리 검증	GTH·MIG·전원·보드 계측	논리 RTL과 라우팅 쿠폰까지만 수행

외부 8GB는 Gemma 1B가 4GB에 들어가지 않아서 선택한 것이 아니다. 더 큰 모델과 긴 문맥으로 확장하거나, 로컬 실행 메모리의 경합을 줄이고, 여러 클러스터에 독립적인 가중치 공급 경로를 제공하기 위한 후보이다. 따라서 외부 구조는 K26-local 기준선보다 나은 조건을 보여야 채택할 수 있다.

IV. K26-Memory FPGA 시스템 구조

K26 측의 입력 명령은 작업 정보와 activation만 포함한다. 명령은 DMA 요청 FIFO를 거쳐 외부 메모리 채널에 가중치를 요청한다. Memory FPGA 경계에서 돌아온 가중치 타일은 작업의 선호 링크에 따라 네 논리 FIFO 중 하나로 들어간다. K26은 job ID로 보관 중인 activation과 응답 가중치를 결합하고, 대기열 정책이 선택한 클러스터에서 MatVec을 수행한다.

새로운 논리 가상 시제품은 작업 수락→DMA 요청→채널 스케줄러→DDR 응답 경계→논리 링크 FIFO→입력 결합→MatVec 결과를 하나의 역압 흐름으로 연결한다. 가중치 응답이 없으면 연산 명령을 만들 수 없고, 동일 job ID의 응답은 한 번만 수락한다. 실제 Gemma 파일에서 제한적으로 추출한 세 개의 16×4 INT8 가중치 타일이 이 경로를 통과해 소프트웨어 INT32 결과와 모두 일치했다.

이 연결은 물리 링크가 완성되었다는 뜻이 아니다. 링크 FIFO는 넓은 논리 응답을 전달하며 GTH 직렬화, lane bonding, CDC, packet 분할, CRC 재전송, credit 반환은 포함하지 않는다. 메모리 응답도 MIG 경계에서 주입하므로 DDR3L PHY와 calibration을 검증하지 않는다. 그림 1은 이 차이를 실선과 점선으로 구분한다.

표 2. 결과를 해석할 때의 증거 구분

구분	포함 내용	허용되는 해석
직접 확인	ONNX 노드·형상, 실제 타일 3개, KiCad 원본	해당 파일과 제한 경로의 사실
RTL 시뮬레이션	달린 논리 경로, exact-once, MatVec 결과	논리 기능과 역압
분석 모델	대기열, 링크·메모리 서비스, 배치 민감도	명시한 가정 안의 상대 비교
미검증	GTH·MIG timing, 보드 전력, SI/PI, 전체 가격	후속 구현·계측 과제

V. Gemma 3 1B 작업의 하드웨어 매핑

로컬에 보관된 Gemma 3 1B ONNX 그래프는 7,837개 노드와 토큰당 183개 dense 투영 연산을 가진다. Attention의 q, k, v, o가 104개, MLP의 gate, up, down이 78개, lm_head가 1개다. 원본 float32 투영 가중치는 약 4.0GB이며, 평가에서는 명시적인 INT8 변환을 가정해 약 1.0GB로 모델링했다. 모델 가중치 자체는 공개 저장소에 배포하지 않는다.

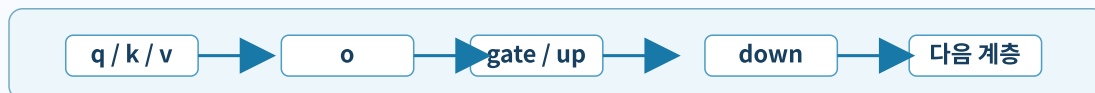
기존 실험은 투영 하나를 작업 하나로 두고 32개 토큰을 동시에 유입했다. 이는 autoregressive decode의 의존성을 위반한다. 개정 모델은 각 투영을 full-K, $N \leq 1024$ 의 출력 타일로 나누어 토큰당 802개 TileJob을 만든다. q/k/v가 끝난 뒤 o, o가 끝난 뒤 gate/up, 이 둘이 끝난 뒤 down을 내보낸다. 다음 layer는 이전 down 뒤에 시작하고, 다음 token은 lm_head가 끝난 뒤에만 시작한다. 32-token 조건은 이 의존성 인식 token을 중첩 없이 직렬 반복한 25,664개 TileJob이다.

실제 그래프 형상을 의존성 있는 출력 타일로 변환

32개 토큰은 겹치지 않으며, 초기 배치는 별도 설계 변수로 비교한다



보수적 단계 장벽



비교한 초기 배치

기준 산술	라운드로빈	크기 균형	채널 친화
464 / 208 / 78 / 52	201 / 201 / 200 / 200	198 / 202 / 200 / 202	394 / 136 / 136 / 136

그림 2. 실제 Gemma 그래프의 투영 연산을 의존성 있는 출력 TileJob으로 변환하는 과정. 배치 규칙은 그래프 사실이 아니라 별도 모델 변수다.

초기 배치는 네 가지다. 기존 산술 규칙은 토큰당 타일 수가 C0/C1/C2/C3에 464/208/78/52로 치우친다. 라운드로빈은 타일 수를 균등하게 나눈다. 크기 균형 배치는 누적 MAC 수가 가장 작은 클러스터에 다음 타일을 둔다. 채널 친화 배치는 가중치가 있는 채널과 같은 번호의 클러스터를 우선한다. 네 배치 모두 실제 compiler 결과가 아니라 설계 변수이며, 이 비교로 “초기 배치를 잘한 효과”와 “작업을 나중에 옮긴 효과”를 분리한다.

VI. 다중 대기열과 지역성 인식 작업 재분배

S1은 각 클러스터가 자기 대기열만 선착순으로 처리하는 정적 기준선이다. S2는 유허 클러스터가 다른 대기열에서 가장 오래 기다린 작업을 가져온다. S3는 기다린 시간에서 가중치 크기, activation 이동, 부분합 반환, 링크 불일치 비용을 뺀 점수가 양수일 때만 작업을 가져온다.

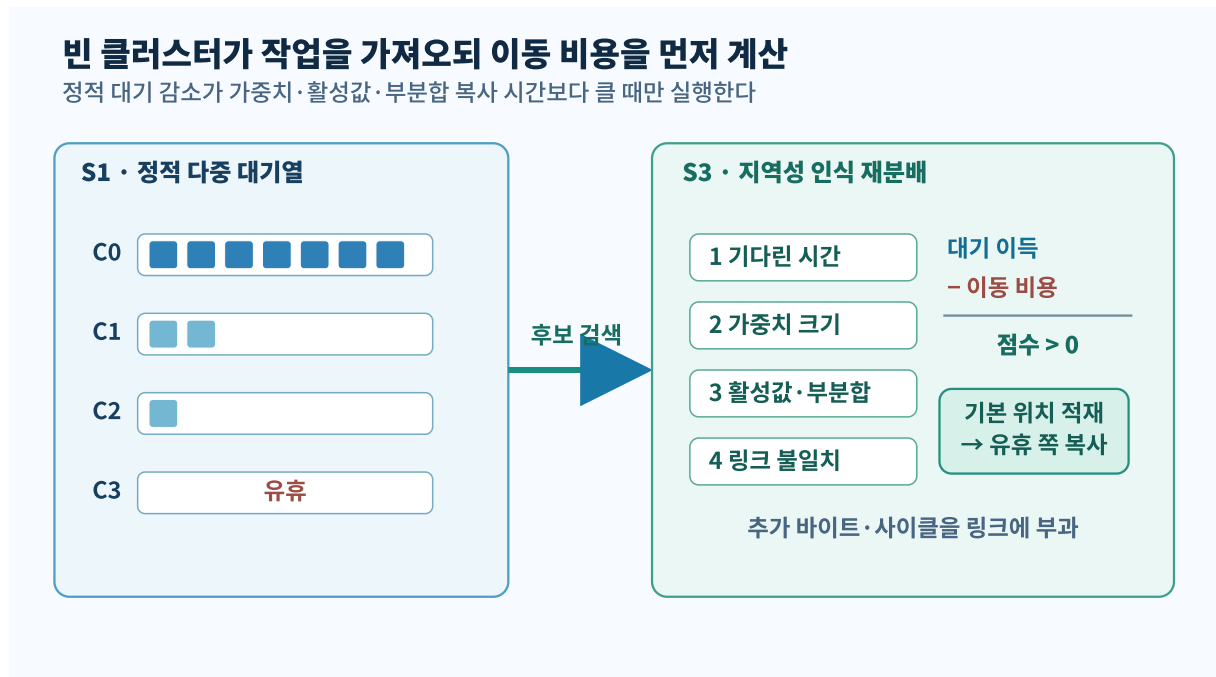


그림 3. 정적 다중 대기열과 지역성 인식 작업 재분배. 훔친 작업은 home prefetch 뒤 thief로 복사되며, 이 추가 byte와 cycle을 링크 서비스에 부과한다.

이번 모델은 작업 훔치기의 비용을 사후 통계로만 세지 않는다. 정적 배치가 선호 링크로 가중치를 먼저 공급했다고 가정하고, 훔친 작업은 가중치·activation·부분합을 실행 클러스터 쪽 링크로 다시 복사한다. 기본 전송과 추가 전송의 byte와 cycle을 분리해 저장한다. 따라서 S3가 더 작은 작업을 고르면 S2보다 비지역 전송량을 줄일 수 있지만, 복사 자체가 정적 S1보다 완료시간을 늘릴 수도 있다.

S3는 상시 정책이 아니다. 균형 배치에서는 옮길 이유가 없고, 메모리 채널 자체가 병목이면 빈 클러스터를 채워도 완료시간이 줄지 않는다. 정책을 켜는 조건은 예상 대기

감소가 추가 데이터 이동 시간보다 큰 경우이다.

VII. 구현과 평가 방법

RTL은 SpinalHDL로 작성하고 Verilator로 시뮬레이션했다. 직접 확인한 경로는 세 실제 가중치 타일의 작업 수락, DMA 명령, DDR 응답 경계, 링크 FIFO, MatVec 결과이다. 별도의 3-job 합성 fixture는 실제 S3 스케줄러가 job ID를 보존하며 세 작업을 훔치고 정확히 한 번 완료하는지 확인한다.

실제 가중치 타일 3개가 페루프 논리 경로를 통과

DMA 명령과 DDR 응답 뒤에만 MatVec이 실행되며 세 결과가 소프트웨어 기준과 일치한다

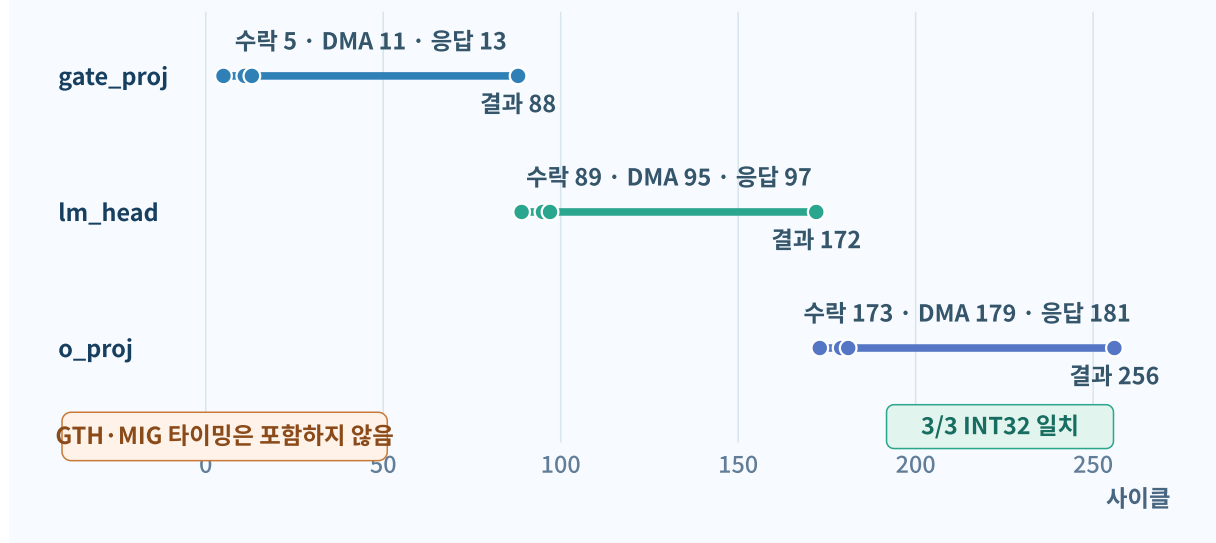


그림 4. 실제 Gemma 대표 타일 세 개가 닫힌 논리 RTL 경로를 통과한 cycle trace. 세 결과는 소프트웨어 INT32 기준과 일치한다.

분석 모델은 4 cluster, 4 memory channel, 4 link bundle, bundle당 128 bit를 기본값으로 사용한다. 합성 부하는 balanced, skew, hotspot, bursty, mixed의 1,000개 작업을 5개 seed에서 생성한다. 같은 작업 부하와 seed의 모든 정책은 동일한 작업 목록 hash를 사용한다. 정책별 중앙값을 나누지 않고 seed별 변화율을 먼저 계산한 뒤 그 중앙값과 범위를 보고한다.

Gemma 평가는 모델에서 추출한 K/N 형상과 보수적 단계 의존성을 사용한다. p95와 p99는 개별 TileJob이 의존성에서 해제된 시점부터 완료될 때까지의 분포이다. 사용자 요청 단위의 꼬리 지연이 아니다. 32-token 시간은 동일한 token 모델의 직렬 반복이며 기능적 텍스트 생성도 아니다.

K26-local 기준선은 외부 링크를 제거하고 공유 로컬 메모리의 유효 대역폭을 4.8, 9.6, 14.4 GB/s로 바꾼다. 외부 후보는 네 채널의 합을 3.2, 6.4, 12.8 GB/s로 바꾸고 링크 폭을 64, 128, 256 bit로 비교한다. 이 값들은 측정값이 아니라 채택 임계값을 찾는 민감도이다.

VIII. 결과와 시스템 설계 결정

A. 합성 불균형 부하

합성 skew에서 S3는 S1보다 TileJob p95를 19.13%, p99를 18.71% 줄였다. 전체 완료시간도 17.25% 짧았다. S2와 비교하면 비지역 가중치 이동량을 35.49% 줄였고 완료시간 중앙값 차이는 +0.01%로 거의 같았다. mixed에서도 S1 대비 p95가 17.70% 줄었다. balanced에서는 변화가 없고 hotspot에서는 p95 변화가 0%였다.

작업 불균형이 큰 합성 부하에서만 꼬리 지연이 감소

S3 대 S1 · 5개 시드의 TileJob p95 변화율 중앙값

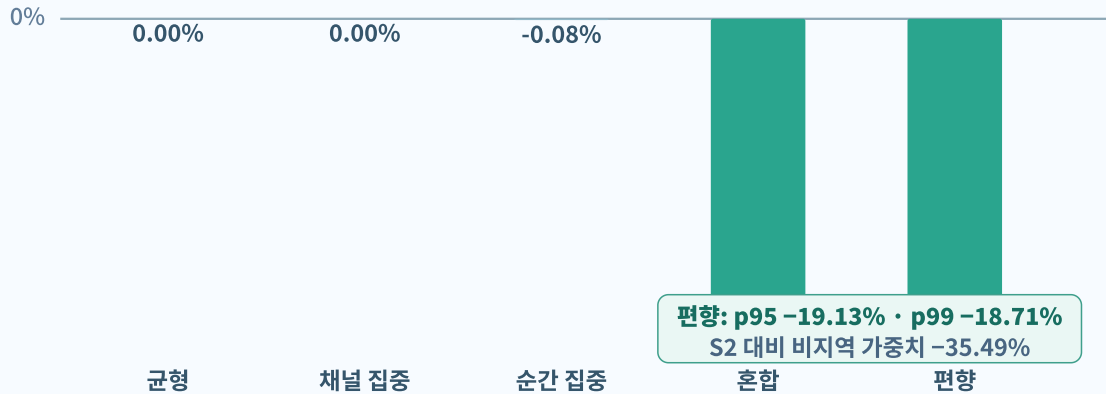


그림 5. 다섯 paired seed의 합성 부하 결과. 작업 불균형이 큰 skew와 mixed에서만 꼬리 지연 감소가 나타난다.

표 3. 합성 부하의 S3 효과, paired seed 중앙값

부하	S1 대비 완료시간	S1 대비 TileJob p95	S1 대비 p99	S2 대비 비지역 가중치
balanced	0.00%	0.00%	0.00%	해당 없음
hotspot	-0.06%	0.00%	0.00%	0.00%
bursty	-1.79%	-0.08%	-1.16%	-15.25%
mixed	-15.91%	-17.70%	-16.92%	-22.60%
skew	-17.25%	-19.13%	-18.71%	-35.49%

이 결과는 작업 불균형이 데이터 이동 비용보다 큰 조건에서만 작업 재분배가 유효하다는 가설을 지지한다. 합성 skew의 수치를 실제 Gemma 실행 결과로

해석해서는 안 된다.

B. Gemma 형상과 초기 배치

Gemma 형상에서는 초기 배치가 결과를 지배했다. 기존 산술 배치에서 S3는 S1보다 완료시간이 4.80% 길고 p95도 0.28% 길었다. 원격 복사 비용을 부과하면 빈 클러스터를 채운 이득보다 이동 비용이 더 컸다. 라운드로빈과 크기 균형 배치에서는 p95가 변하지 않았고 완료시간만 1% 미만 줄었다. 채널 친화 배치에는 큰 작업 쏠림이 남아 S3가 완료시간을 7.56%, p95를 19.79% 줄였다.

같은 Gemma 형상도 초기 배치에 따라 결과가 달라진다

S3 대 S1 · TileJob p95와 전체 완료시간 변화율

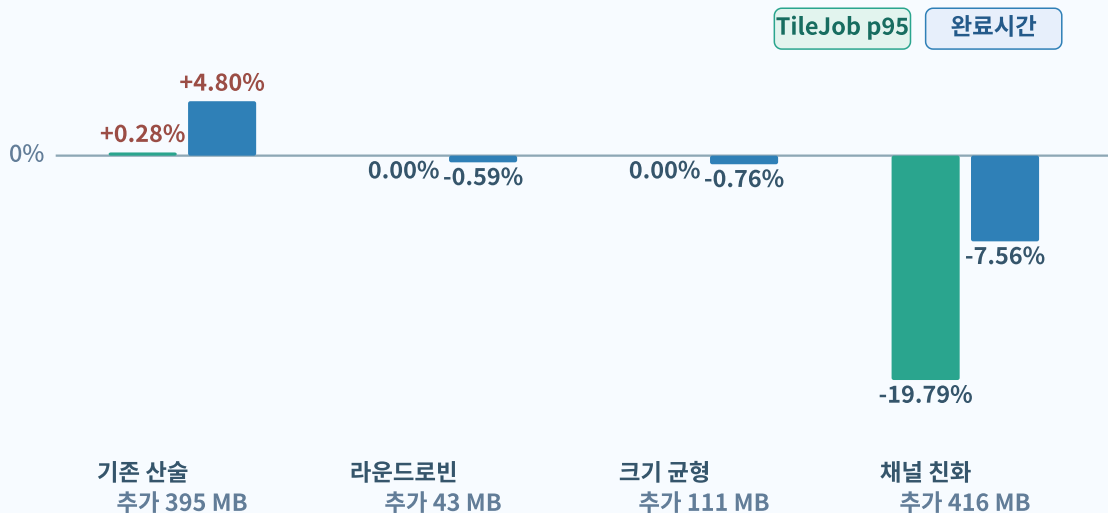


그림 6. 같은 Gemma 형상에서 초기 배치에 따라 S3의 효과가 바뀐다. 배치가 이미 균형이면 꼬리 지연 이득이 없다.

표 4. Gemma dependency-aware TileJob의 S3 효과

초기 배치	타일 수 C0/C1/C2/C3	완료시간 변화	TileJob p95 변화	추가 링크 전송
기존 산술 규칙	464/208/78/52	+4.80%	+0.28%	395.2 MB/token
라운드로빈	201/201/200/200	-0.59%	0.00%	43.3 MB/token
크기 균형	198/202/200/202	-0.76%	0.00%	110.8 MB/token
채널 친화	394/136/136/136	-7.56%	-19.79%	416.4 MB/token

결론은 “Gemma에서 작업 흠치기가 빠르다”가 아니다. 초기 배치가 충분히 균형이면 작업을 옮길 필요가 없다. 채널 지역성을 우선해 큰 불균형이 남은 경우에만 선택적 재분배가 의미가 있다.

C. K26-local과 외부 메모리 후보

크기 균형 S1에서 K26-local 4.8/9.6/14.4 GB/s의 완료시간은 각각 45.0M/24.6M/17.8M cycle이었다. 외부 4채널 3.2/6.4/12.8 GB/s 후보는 238.3M/120.6M/62.4M cycle이었다. 외부 6.4 GB/s 조건은 링크 폭을 64 bit에서 256 bit로 넓혀도 거의 바뀌지 않아 메모리 공급이 병목이었다.

이 민감도에서는 외부 후보가 Gemma 1B의 성능 기준선을 이기지 못한다. 또한 INT8 context-32K 용량 2.43 GiB가 로컬 4GB 안에 들어가므로 용량 채택 근거도 없다. 외부 Memory FPGA는 3B·더 긴 문맥·실제 로컬 경합을 평가할 다음 구조 후보이며, Gemma 1B의 기본 구성은 K26-local이다.

비용 자료는 DDR3L 부품 가격 snapshot만 포함하고 K26, FPGA, PCB, 전원, 냉각, 조립을 제외한다. 에너지도 MAC·링크·DRAM 동적 항의 민감도일 뿐이다. 따라서 저비용·저전력은 설계 목표이지 이번 연구가 실물로 달성한 결과가 아니다.

IX. PCB 인터페이스 라우팅 쿠폰

KiCad 자료는 전체 Memory FPGA 보드가 아니라 인터페이스 배선의 공간을 검토한 쿠폰이다. J1은 XC7K160T의 DDR/GTH bank 경계를, J2는 K26 GTH 경계를 대신한 범용 Samtec 커넥터다. U1은 네 메모리 채널 중 DDR3L x16 한 슬라이스만 나타낸다. 실제 K26 SOM 커넥터 핀과 FPGA BGA ball, 네 MIG 채널, 전원 트리, 부트 회로는 포함하지 않는다.

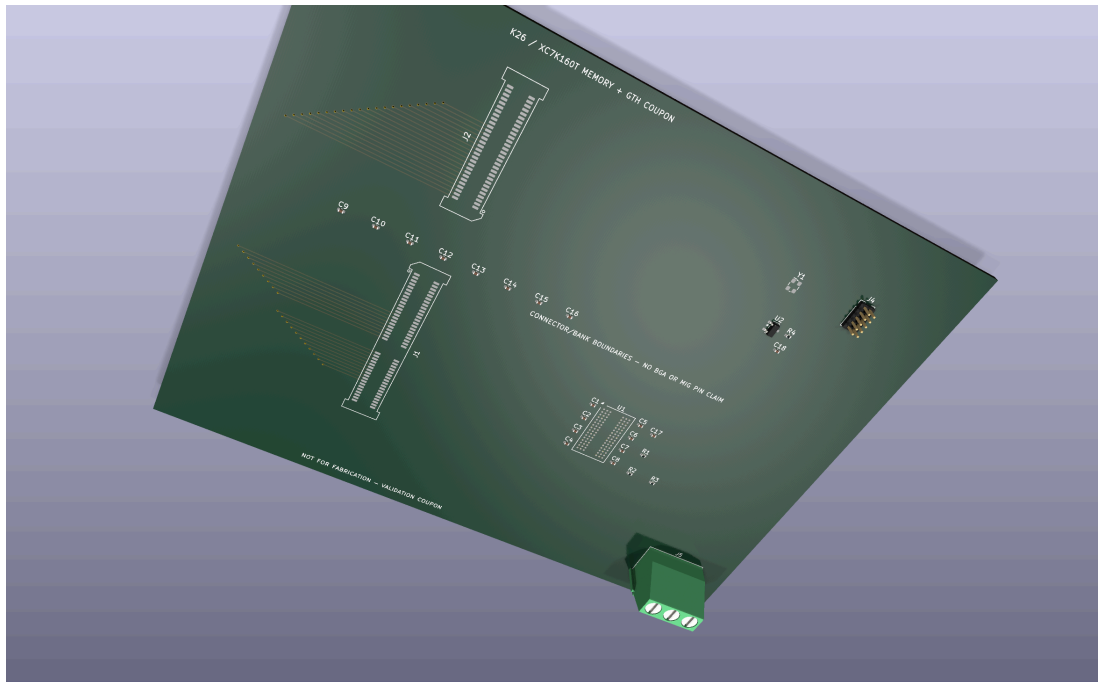


그림 7. 실제 KiCad 원본에서 렌더링한 인터페이스 라우팅 쿠폰. 두 범용 경계 커넥터, 대표 DDR3L x16 슬라이스, GTH·refclk 배선을 표시한다. NOT FOR FABRICATION.

쿠폰은 29개 footprint와 20개 routed GTH/refclk net을 포함하며 선언된 제한 범위의 ERC/DRC는 0건이다. 그러나 전체 116개 net 중 55개가 미배선이고, high-speed return path와 ground stitching, test point, 정확한 부품 출처가 완성되지 않았다. 이 자료가

지지하는 주장은 “대표 링크와 DDR 배선의 위치 관계를 KiCad 객체로 구체화했다”는 것뿐이다. 제작 가능성, SI/PI, 온도, 실제 대역폭은 검증하지 않았다.

X. 논의와 한계

첫째, Gemma 작업의 K/N 형상과 투영 순서는 실제 모델에서 얻었지만 초기 cluster/channel/bundle 배치는 모델 변수다. 실제 compiler나 runtime 배치가 생기면 네 민감도 중 어느 조건에 가까운지 다시 확인해야 한다. 둘째, 단계 장벽은 잘못된 병렬 실행을 막는 보수적 모델이며 실제 연산자의 세부 overlap을 재현하지 않는다. 셋째, p95와 p99는 TileJob 분포이지 사용자 요청 분포가 아니다.

넷째, 닫힌 RTL 경로는 논리 가상 시제품이다. GTH·MIG·CDC·packet protocol을 구현하고 Vivado timing을 달아야 물리 bandwidth를 말할 수 있다. 다섯째, K26-local 대역폭과 전력은 민감도 입력이다. 실제 보드에서 같은 작업을 실행해 로컬 경합과 전체 전력을 측정해야 외부 메모리 채택을 결정할 수 있다.

후속 검증 순서는 명확하다. 먼저 실제 K26-local 대역폭·경합·전력을 측정한다. 다음으로 논리 응답을 packet 단위로 나누고 GTH·CDC·credit 경로를 구현한다. 그 뒤 MIG timing과 한 채널의 정확한 핀·전원·배선을 달고 SI/PI 검증을 진행한다. 이 관문을 통과하기 전에는 전체 보드를 제작 대상으로 취급하지 않는다.

XI. 결론

본 연구는 K26의 연산·제어와 외부 Memory FPGA의 가중치 공급을 분리한 온디바이스 sLLM 확장 후보를 설계하고, 실제 Gemma 형상에서 초기 배치와 작업 재분배의 조건을 평가했다. 합성 불균형 부하에서는 지역성 인식 작업 재분배가 TileJob p95와 비지역 가중치 이동을 함께 줄였다. 그러나 의존성과 이동 비용을 반영한 Gemma 형상에서는 배치에 따라 이득이 사라지거나 완료시간이 악화되었다.

따라서 Work Stealing은 이 구조의 주인공이 아니라, 초기 배치 후 남은 불균형이 이동 비용보다 클 때만 켜는 수단이다. Gemma 1B는 K26-local을 우선하고, 외부 8GB는 더 큰 모델·긴 문맥·실제 로컬 경합이 확인될 때 채택한다. 실제 모델 그래프, 출력 TileJob, 닫힌 논리 RTL, 분석 결과, KiCad 쿠폰을 공개한 설계 흐름이 본 연구의 핵심 공학적 기여다.

코드와 자료 공개

SpinalHDL RTL, 실험 설정과 결과 CSV, Gemma 그래프 manifest, Figure 원본, KiCad 원본, 논문과 발표자료는 다음 저장소에 공개한다.

Repository:

<https://github.com/snowman0919/varp-k26-memory-fpga>

Release: <https://github.com/snowman0919/varp-k26-memory-fpga/releases/tag/v1.1-conference-final>

모델 가중치는 라이선스와 용량 때문에 배포하지 않는다. 공개 archive는 고정 hash의 그래프 manifest와 제한된 타일 fixture만 포함한다.

참고문헌

- [1] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” arXiv:2309.06180, 2023.
- [2] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” arXiv:2205.14135, 2022.
- [3] E. Frantar et al., “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” arXiv:2210.17323, 2022.
- [4] G. Xiao et al., “SmoothQuant: Accurate and Efficient Post-Training Quantization for LLMs,” arXiv:2211.10438, 2022.
- [5] J. Lin et al., “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” arXiv:2306.00978, 2023.
- [6] B. Li et al., “FTRANS: Energy-Efficient Acceleration of Transformers using FPGA,” ISLPED, 2020.
- [7] S. Hong et al., “DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation,” MICRO-55, 2022.
- [8] S. Zeng et al., “FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs,” arXiv:2401.03868, 2024.
- [9] S. Li et al., “DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator,” IEEE Computer Architecture Letters, vol. 19, no. 2, 2020.
- [10] AMD, Kria K26 SOM Data Sheet DS987.
- [11] AMD, Kria SOM Carrier Card Design Guide UG1091.
- [12] AMD, 7 Series FPGAs Packaging and Pinout UG475.
- [13] AMD, 7 Series FPGAs Memory Interface Solutions UG586.
- [14] KiCad Project, KiCad 9 Documentation, ERC and DRC reference.
- [15] R. D. Blumofe and C. E. Leiserson, “Scheduling Multithreaded Computations by Work Stealing,” Journal of the ACM, vol. 46, no. 5, pp. 720–748, 1999.
- [16] Q. Chen et al., “Locality-Aware Work Stealing Based on Online Profiling and Auto-Tuning for Multisocket Multicore Architectures,” HPDC, 2015.